

Rozdělení na dvě role: **Analytik logiky (Student A)** a **Procesní integrátor (Student B)**.

Oba budete mít zhruba stejný objem psaní dokumentace (teorie) i programování (kódu).

Student A: Analytik logiky a Unit tester

Tato role se zaměří na detailní pravidla aplikace, matematiku na pozadí a čisté testování funkcí bez závislostí. Pracovat bude primárně s metodou `validateReservation`.

Teoretická část (Dokumentace):

- **Testy vstupů:** Výběr 2 netriviálních vstupů (např. parametr `people` a časové rozmezí/`duration`).
- **EC a BVA:** Vypracování tabulek tříd ekvivalence a hraničních hodnot pro tyto vybrané vstupy.
- **Pairwise testing:** Vytvoření matice kombinací testovacích dat pro validaci.
- **Detailní scénář (1 ze 2):** Sepsání jednoho detailního testovacího scénáře (např. "Pokus o vytvoření rezervace mimo pracovní dobu s překročenou kapacitou").

Praktická část (Kód):

- **Unit testy (10 testů):** Napsání 10 čistých JUnit testů, které otestují všechny výjimky a pravidla v netriviální metodě `validateReservation`.

Student B: Procesní integrátor a Mockito tester

Tato role se zaměří na to, jak systém funguje jako celek, jak se mění stavy rezervací a jak komunikují jednotlivé vrstvy (`Controller` -> `Service` -> `Repository`).

Teoretická část (Dokumentace):

- **Testy průchodů (Diagramy):** Nakreslení UML diagramů pro 2 procesy aplikace (např. proces createReservation a proces cancelReservation).
- **Procesní testy (TDL 2):** Návrh testů pokrývajících všechny přechody ve vytvořených diagramech (např. z CREATED do CANCELLED).
- **Detailní scénář (1 ze 2):** Sepsání druhého detailního testovacího scénáře (např. "Úspěšné vytvoření rezervace a její následné zrušení uživatelem").

Praktická část (Kód):

- **Testy s Mockito (5 testů):** Napsání 5 unit testů pro ReservationService, kde se pomocí Mockita nasimuluje chování databáze (např. simulace konfliktu časů přes RoomRepository a ReservationRepository).
- **Integrační testy (8 testů):** Napsání 8 procesních testů pomocí anotace @SpringBootTest, které projdou celou aplikací od založení pokoje až po uložení do In-Memory databáze.

Proč je toto rozdělení spravedlivé a efektivní?

1. **Rovnováha dokumentace:** Student A dělá složitější analytické tabulky (EC, BVA, Pairwise). Student B zase musí vymyslet a graficky nakreslit dva procesní diagramy.
2. **Rovnováha kódu:** Student A píše 10 klasických Unit testů. Student B píše sice 13 testů, ale 8 z nich jsou integrační, které se po úvodním nastavení Spring Boot kontextu píšou velmi rychle, a zbylých 5 využívá Mockito.
3. **Minimální konflikty při slučování (Merge conflicts):** Student A bude upravovat převážně testovací třídu pro validace. Student B bude pracovat v testovací třídě s Mockitem a v integračních testech. Nebudete si tak přepisovat stejný kód.